13. I/O: io, Files, Readers and Writers

13. I/O: io, Files, Readers and Writers

Phase: Phase 2 — Core language depth
Estimated time: 1–2 days
Slug: `13-io-readers-writers`

Learning objectives

- Use io.Reader and io.Writer as the core I/O abstractions
- Stream data without loading entire files when possible
- Compose readers (bufio, limit, multi)
- Always close resources with defer carefully

Prerequisites

Complete prior numbered lessons. Type every example yourself.

The io contracts

```
type Reader interface { Read(p []byte) (n int, err error) }
type Writer interface { Write(p []byte) (n int, err error) }
```

Almost all I/O in Go is streaming through these interfaces — files, sockets, buffers, compression.

```
f, err := os.Open(path)
if err != nil { return err }
defer f.Close()
return io.Copy(dst, f)
```

bufio

Wrap expensive readers/writers with buffering for small reads/writes.

Errors

`io.EOF` signals end of stream — often expected, not always a hard failure. `io.ErrUnexpectedEOF` means truncated data.

Common mistakes (especially from Python instincts)

- Loading multi-GB files into memory with os.ReadFile without need
- Forgetting defer Close
- Ignoring partial Write/Read return values

Exercises

Work in `exercises/13-io-readers-writers/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.

- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/13-io-readers-writers/` until you have an honest failing attempt.

Self-check

1. Why is io.Reader more powerful than “give me []byte”?
2. What does io.Copy do?
3. When is io.EOF expected?

Further reading

- https://pkg.go.dev/io
- https://go.dev/blog/laws-of-reflection